COSTRUIRE UN GIOCO CON PIÙ STAGE

Mappe e livelli: stage, spritesheet e livelli 3D

Spritesheet, flag delle tile, layer della mappa, strumenti lineari e mappe 3D: dalla prima tile all'ultimo stage.

Indice

Cos'è un livello in bm **5**

Cosa fa 5

Perché conta 5

Da sapere 5

Attenzione 5

Lo sprite sheet e le tile **7**

Da dove partire 7

Disegna le tile 7

I flag di una tile 7

Zone e box 8

La mappa 2D in bm SDK **9**

Cosa fa 9

Come aprirla 9

I tasti della pagina mappa 9

Layer 9

Dipingi uno stage 10

I flag nel codice 10

Gli strumenti lineari (percorso, mele, casse, lampade) **11**

Cosa fa 11

Gli strumenti 11

Da sapere 11

Disegna un percorso 11

Livelli 3D **13**

Cosa fa 13

I quattro layer 13

Da sapere 13

I segnaposto 13

L'anteprima 3D 14

Costruisci un piccolo livello 3D 14

Attenzione . . . 14

Costruire un gioco con la suite . . . 15

Il percorso . . . 15

Il progetto e la build . . . 15

I file nella cartella di un gioco . . . 15

Attenzione . . . 15

Più stage in un gioco 2D . . . 16

Cosa fa . . . 16

Tre modi per costruire gli stage . . . 16

Da sapere . . . 16

Crea tre stage con i layer . . . 16

Regioni in una mappa grande . . . 16

Attenzione . . . 17

Più stage in un gioco 3D . . . 18

Cosa fa . . . 18

Da sapere . . . 18

Le funzioni del gioco . . . 18

Attenzione . . . 18

Uno stage 3D nel codice . . . 18

Passare da un livello all'altro nel gioco . . . 20

Cosa fa . . . 20

Da sapere . . . 20

Un gioco con tre stage . . . 20

Provalo, passo per passo . . . 21

Attenzione . . . 21

Consigli e limiti . . . 22

Limiti . . . 22

Consigli . . . 22

Quando qualcosa va storto . . . 22

Cosa non c'è ancora . . . 22

Punti aperti **23**

LEGGIMI

Cos'è. La guida per costruire i **livelli** di un gioco con la suite bm: come stanno insieme sprite sheet e mappa, come disegnare una mappa 2D, come costruire un livello 3D e come un gioco passa da uno stage al successivo.

Come usarla. Leggi una volta il primo capitolo: spiega cosa bm ti dà per i livelli e cosa lascia al tuo codice. Poi segui i passi numerati del capitolo che ti serve. Ciò che sta in un riquadro con bordo e ombra è fatto per essere copiato così com'è.

Cosa manca. Alcune cose in bm non ci sono ancora, o ci sono in parte. Sono segnate **[IN SVILUPPO]** con una riga che dice cosa manca. Non si inventa nulla.

Cos'è un livello in bm

Un gioco bm è una cartuccia: uno sprite sheet, una mappa fatta di layer, modelli 3D facoltativi e il codice. bm non ha un "file di livello" separato. Un livello è ciò che il tuo codice decide di mostrare dalla mappa e dallo sheet.

Cosa fa

- Lo **sprite sheet** è l'unica immagine con tutte le tile e gli sprite. Lo disegni in bm Pixel.
- La **mappa** è una griglia di numeri di cella che puntano alle tile dello sheet. Ha da 1 a 8 **layer** della stessa dimensione, ciascuno con un nome.
- I **flag** (da 0 a 7) appartengono a ogni tile dello sheet e dicono al gioco cos'è una tile: muro, piattaforma, scala, acqua, pericolo.
- I **box** sono rettangoli salvati con le zone con nome dello sheet: dove un personaggio può essere colpito e dove il suo colpo fa male.
- Per il **3D**, quattro layer speciali (`floor` , `height` , `walls` , `objects`) diventano geometria vera che il gioco disegna e con cui collide.

Perché conta

Costruisci il mondo sulla console stessa, con bm SDK: disegni le tile, dipingi la mappa, premi F5 e giochi. Lo stesso file diventa il `.bm` finale con Ctrl+B.

Da sapere

- Gli editor lavorano su un **progetto** (`.bme`). Un gioco (`.bm`) è di sola lettura; salvandolo si crea una copia modificabile.
- Una mappa nuova è di 256x256 celle con un layer.
- Le sezioni della cartuccia che qui contano sono 11 SPRITES (zone), 12 LAYERS (mappa), 13 FLAGS (flag delle tile) e 14 BOXES (hitbox).
- Più stage li fa il tuo codice: sceglie quale parte o quale layer della mappa mostrare e cosa metterci. I capitoli 7 e 9 mostrano come.

Attenzione

- **[IN SVILUPPO]** Non esiste un elenco di livelli o stage dentro la cartuccia, con un tasto per passare dall'uno all'altro nell'editor. Cosa manca: bm SDK ha una sola mappa e nessuna voce "livello 1, livello 2". Oggi gli stage sono layer o regioni della stessa mappa, oppure li costruisce il tuo codice.

- **[IN SVILUPPO]** `lib.world()` (un caricatore di mappa, entità e trigger) e una griglia di stanze per la pagina mappa sono solo proposte nelle note di progetto di bm SDK. Nella suite non ci sono.
- **[IN SVILUPPO]** L'importazione di mappe da Tiled non c'è. Cosa manca: tutta l'importazione; la mappa viene dall'SDK o da file CSV.

Lo sprite sheet e le tile

Da dove partire

Lo sheet si disegna in **bm Pixel**: aprilo dalla scheda **Dev** del menu, oppure premi **F3** in bm SDK per vedere gli sprite. Lo sheet è un'immagine divisa in celle da 8x8. La cella `n` sta alla colonna `n % (width/8)` e alla riga `n // (width/8)`. Può arrivare a 4096x4096 pixel.

Disegna le tile

1. Apri il progetto in bm Pixel.
2. Premi **z** finché la dimensione dello sprite è 8. Ora uno sprite è una tile.
3. Disegna una tile di pavimento, una di muro e una di scala, ciascuna nella sua cella. Tieni vicine quelle dello stesso tipo.
4. Premi **F2** per vedere l'intero sheet e controllare dov'è ogni tile.
5. Premi **Ctrl+S** per salvare.

> **NOTA**
> Disegna le tile a dimensione 8. Una tile 16x16 sono solo quattro celle, che l'editor di mappa può piazzare insieme (vedi i capitoli seguenti).

I flag di una tile

Ogni tile ha 8 flag, numerati da 0 a 7. Il gioco legge i flag e non i numeri delle tile, così puoi cambiare il disegno senza toccare il codice. La convenzione di bmlib, che ogni gioco può cambiare:

Flag	Valore	Significato
0	1	solido
1	2	piattaforma: si attraversa dal basso
2	4	scala
3	8	acqua
4	16	fa male
5 - 7		liberi

I flag si impostano in **bm SDK**, non in bm Pixel:

1. Apri il progetto in bm SDK e premi **F3**: la pagina degli sprite.
2. Scegli la cella della tile del muro.
3. Premi **0** per accendere il flag 0. I tasti da **0** a **7** sono i flag della cella.
4. Scegli la cella della scala e premi **2**.
5. Premi di nuovo **F3** per la mappa e premi **c**: le tile con flag si mostrano con una cornice colorata.

NOTA

I flag appartengono alla tile dello sheet, non a un punto della mappa. Ogni layer e ogni stage che usa quella tile ottiene gli stessi flag.

Zone e box

Una **zona** dà un nome a un rettangolo dello sheet, con i suoi fotogrammi e la velocità dell'animazione. I **box** sono i rettangoli dei fotogrammi di una zona: `hurt` dove può essere colpita, `hit` dove il suo colpo fa male, `body` lo spazio che occupa.

- Zone e box si scrivono in un file di testo (`zones.txt`): una zona è `nome x y larghezza altezza [fotogrammi [fps]]` , i suoi box sono le righe sotto. Il file entra nella cartuccia con `mkbm.py --sprites` . bm Pixel mostra i box sopra lo sprite (tasto **j**).
- Il gioco li legge con `zspr()` , `zone()` e `zboxes()` .
- **[IN SVILUPPO]** Dare un nome a una zona e disegnare, spostare o ridimensionare un box sulla console non è possibile. Cosa manca: un editor di zone e box; oggi si scrivono nel file di testo delle zone.

DA COPIARE

```
punch   0  64 32 32 4 12
  hurt  *  8  2 16 30
  hit   3  24 10 10  6
```

La mappa 2D in bm SDK

Cosa fa

La pagina mappa di bm SDK dipinge la mappa con le tile dello sheet, su fino a 8 layer. Mostra il layer in cui sei, la cella sotto il cursore e la tile che tieni.

Come aprirla

Premi **F3** per vedere gli sprite, poi **F3** di nuovo: la mappa. Premi ancora **F3** per tornare agli sprite. Tieni premuto **F12** per vedere i tasti della pagina.

I tasti della pagina mappa

Tasto	Azione
frecce	muovono il cursore
PgUp / PgDn, Home / End	una pagina su / giù, a sinistra / a destra
Spazio	piazza la tile
Backspace	svuota la cella
x	prende la tile sotto il cursore
, e .	la tile prima / dopo
Tab	sceglie la tile
f	riempie
z	8x8 o 16x16 (una tile di 2 x 2 celle, sulle celle pari)
+ / -	ingrandisce / riduce
m	l'intera mappa 256x256 in una schermata (panoramica)
l	il layer successivo
Shift+L	un nuovo layer
r	rinomina il layer
o	solo questo layer, oppure tutti
c	mostra i flag delle tile
u	annulla

Con il mouse: il tasto sinistro piazza la tile, quello centrale la prende, la rotella scorre.

Layer

- Una mappa ha al massimo 8 layer: l'SDK dice `a map has at most 8 layers`.
- Il primo layer è disegnato sul fondo. I nomi sono tuoi: `back`, `front`, `stage2`.
- Tutti i layer hanno la stessa dimensione.
- La riga di stato mostra il layer e il suo nome.

NOTA

In un layer chiamato `objects` il pennello diventa 16x16 e viene scelta la prima tile segnaposto. I segnaposto sono numeri da 1024 in su: il gioco non li disegna, li legge e costruisce cose in quel punto.

Dipingi uno stage

1. Apri un progetto da un modello, per esempio **Platform 2D** (**Ctrl+N**), o il tuo.
2. Premi **F3** due volte per vedere la mappa.
3. Premi **Shift+L** per aggiungere un layer. Premi **r** e scrivi il nome `stage1`.
4. Premi **Tab**, scegli una tile di pavimento con le frecce e premi **Invio**.
5. Muoviti con le frecce e premi **Spazio** per dipingere il terreno.
6. Scegli una tile di muro, dipingi i muri, poi premi **c** per controllare i flag: le tile solide mostrano una cornice.
7. Premi **m** per vedere l'intera mappa, poi di nuovo **m** per tornare.
8. Premi **u** se sbagli.
9. Premi **Ctrl+S** per salvare, poi **F5** per provare il gioco.

PROVA

Aggiungi un secondo layer chiamato `front`, dipingici cime di alberi o archi, e premi **o** per guardare solo questo layer e poi tutti.

I flag nel codice

Il gioco legge i flag con `fget`, `mget` e `mflags`.

DA COPIARE

```
-- la cella è solida
if fget(mget(cx, cy), 0) then end

-- il personaggio (8x8) poggia su qualcosa di solido
if mflags(px, py + 8, 8, 1) & 1 ~= 0 then on_ground = true end

-- disegna solo le scale del layer 1
map(0, 0, 0, 0, 80, 45, 1, 4)
```

Gli strumenti lineari (percorso, mele, casse, lampade)

Cosa fa

Nella pagina mappa, alcuni strumenti ti evitano di piazzare ogni cella a mano. Piazzano una linea di tile o un oggetto alla volta. Sono pensati per le mappe 3D, dove lavorano sui layer `floor` e `objects`. Il capitolo 5 li usa in un livello completo.

Gli strumenti

Tasto	Strumento	Cosa fa
Shift+P	percorso	un corridoio da una cella di partenza a una di arrivo, sul layer `floor`, con la tile che tieni
Shift+W	larghezza	cambia la larghezza del corridoio, da 1 a 5
Shift+A	fila di mele	una linea di segnaposto mela (1026) sul layer `objects`
Shift+C	cassa	un segnaposto cassa (1028) sul cursore, sul layer `objects`
Shift+O	lampada	un segnaposto lampada (1032) sul cursore, sul layer `objects`
Shift+S	partenza	il segnaposto di partenza (1024) del livello
Shift+G	arrivo	il segnaposto di arrivo (1030) del livello

Da sapere

- Percorso e mele richiedono due pressioni di **Invio**: una sulla cella di partenza, una su quella di arrivo. La linea viene disegnata tra le due. Una linea gialla la mostra mentre ti muovi.
- Gli strumenti cercano un layer chiamato `floor` o `objects`. Se manca, dipingono nel layer in cui sei.
- Con il pennello 16x16 attivo (**z**) la linea avanza a passi di due celle.
- Un solo **u** annulla un intero percorso, una fila o un oggetto.
- Gli strumenti sono nel menu della pagina (il tasto destro del mouse).

Disegna un percorso

1. Premi **F3** due volte. Scegli una tile di pavimento con **Tab**.
2. Premi **Shift+W** finché il messaggio dice `path width: 3`.
3. Premi **Shift+P**: il messaggio dice `path tool (width 3): press Enter at start cell`.
4. Vai alla cella di partenza e premi **Invio**.

5. Vai alla cella di arrivo e premi **Invio**. Il messaggio dice `path drawn (width 3)` .
6. Premi **u** per togliere l'intero corridoio.

Livelli 3D

Cosa fa

Un livello 3D è una mappa i cui layer vengono letti dal runtime e trasformati in pavimenti, muri e oggetti. Lo stesso costruttore è usato dall'anteprima dell'editor e dal tuo gioco, quindi ciò che vedi è ciò che il gioco ottiene.

I quattro layer

Premi **T** nella pagina mappa (o *Template: 3D map* nel menu) e la mappa riceve questi layer:

Layer	Cosa contiene
`floor`	la tile del terreno; una cella vuota è un buco
`height`	l'altezza del terreno, come numero
`walls`	l'altezza di un blocco di muro sopra il terreno
`objects`	segnaposto: partenza, mele, casse, arrivo, lampade

Il modello attiva anche il pennello 16x16.

Da sapere

- Una cella della mappa 3D è un blocco di 2x2 tile. Il costruttore legge i layer nella **tile in alto a sinistra** di ogni blocco. Un pennello 16x16 con la tile 0 dà un pavimento di buchi.
- Un numero di altezza è moltiplicato per il passo (0.25 di default) e dà l'altezza in unità del mondo. Un numero di `walls` vale lo stesso, sopra il terreno.
- L'editor disegna un modello al posto dell'oggetto se la cartuccia ha un modello con il nome dell'oggetto (`APPLE` , `CRATE` , `LAMP` , `GOAL` , `PICO`). Senza, mostra un punto colorato; le lampade danno una luce.
- Le luci delle lampade del gioco sono al massimo 4 (slot `lamp3d` da 1 a 4).

I segnaposto

Numero	Nome nel gioco
1024	`PLAYER_START`
1026	`APPLE`
1028	`CRATE`
1030	`GOAL`
1032	`LAMP`
1034	`CRATE_BOUNCE`

Il selettore delle tile si ferma all'ultima tile dello sheet, quindi i marcatori hanno tasti propri: **Shift+S**, **Shift+G**, **Shift+A**, **Shift+C**, **Shift+O**. **[IN SVILUPPO]** `CRATE_BOUNCE` (1034) è noto al runtime ma non ha un tasto nell'editor: scrivilo dal codice con `mset` .

L'anteprima 3D

Premi **v** nella pagina mappa per passare tra la mappa 2D e l'anteprima 3D.

- sinistra / destra girano la camera; su / giù la avvicinano e la allontanano.
- Shift+Su e Shift+Giù la inclinano.
- `,` e `.` cambiano la tile che tieni.
- Il quadrato giallo sul terreno è il cursore; una linea gialla mostra uno strumento in uso.

La barra di stato conta mele, casse e lampade e dà una stima del numero di triangoli per pavimento e oggetti. Usala come budget mentre costruisci.

Costruisci un piccolo livello 3D

1. Premi **Ctrl+N** e scegli un modello 3D, oppure parti dal tuo progetto.
2. Premi **F3** due volte, poi **T**: compaiono i quattro layer.
3. Scegli il layer `floor` con **I**. Scegli una tile di pavimento con **Tab**.
4. Premi **Shift+P**, **Invio** a un'estremità, **Invio** all'altra: un corridoio. Ripeti per fare una stanza.
5. Scegli il layer `height` con **I** e piazza una tile: il suo numero è l'altezza del blocco, in passi. Scegli `walls` e fai lo stesso per un muro.
6. Premi **v**: vedi il livello. Orbita con le frecce.
7. Premi **Shift+S** dove parte il giocatore e **Shift+G** all'arrivo. I marcatori vanno da soli nel layer `objects`.
8. Premi **Shift+A**, **Invio**, **Invio** per una fila di mele, **Shift+C** per una cassa, **Shift+O** per una lampada.
9. Controlla la stima dei triangoli nella barra di stato. Premi **Ctrl+S**, poi **F5**.

ATTENZIONE

Il costruttore legge sempre il layer chiamato `objects` per i segnaposto. Non rinominarlo. I layer `floor`, `height` e `walls` possono avere altri nomi, che passi a `map3d_build`.

Attenzione

- **[IN SVILUPPO]** Le mappe 3D di bm SDK non sono ancora chiuse. Cosa manca: la verifica che una mappa 3D si salvi e si ricarichi. Previsto per la beta (rev 1).

Costruire un gioco con la suite

Il percorso

1. **bm Pixel**: disegna lo sheet.
2. **bm SDK**, pagina sprite (F3): imposta i flag delle tile.
3. **bm SDK**, pagina mappa (F3 due volte): dipingi i layer.
4. **bm Studio** e **bm Mesh**: crea gli oggetti 3D (modelli) se il livello è 3D. La cartuccia li tiene nella sezione MESH.
5. **bm SDK**, pagina codice (F2): scrivi la logica degli stage.
6. **F5**: prova il gioco; torni alla stessa pagina.
7. **Ctrl+B**: costruisci il `.bm` finale.

Il progetto e la build

- Il progetto è un file `.bme`: sheet, mappa con i layer, flag, zone, box, modelli e il codice. Salvalo con **Ctrl+S**.
- **Ctrl+B** costruisce il `.bm` del progetto (o il `.b16` per l'RGB30, secondo il target che l'SDK mostra). Il `.b16` è un'edizione separata con la sua build.
- Sheet, mappa e flag entrano nella cartuccia nelle sezioni 11, 12, 13 e 14, quindi `cart_load` e `cart_save` li portano con sé.
- Per un gioco che vive nel repository, la build prende dei file: `mkbm.py --sheet sheet.png --map map.csv`, e un layer in più per ogni `--map nome=file.csv` (fino a 8, nell'ordine dato).

DA COPIARE

```
python3 scripts/mkbm.py --sheet sheet.png --map map.csv --map front=map_front.csv --flags flags.csv --sp
```

I file nella cartella di un gioco

File	Cos'è
`sheet.png`	lo sprite sheet
`map.csv`	il primo layer
`map_<name>.csv`	un layer in più, di nome `<name>`
`flags.csv`	i flag: numeri da 0 a 255 per le celle 0, 1, 2... oppure `n=flag` per la cella `n`
`sprites.txt`	le zone e i loro box

Attenzione

- Un kernel più vecchio dei layer disegna il primo layer e ignora gli altri.
- **[IN SVILUPPO]** Un gioco non si può ancora sviluppare del tutto senza PC: alcuni file di risorse sono solo da PC. Cosa manca: gestire i file di risorse sul dispositivo. Previsto per la beta (rev 1).

Più stage in un gioco 2D

Cosa fa

Il tuo gioco attraversa degli stage. bm ti dà i pezzi; il codice sceglie quale è sullo schermo. Ci sono tre modi, e si possono mescolare.

Tre modi per costruire gli stage

Modo	Come	Adatto a
Un layer per stage	`stage1` , `stage2` , `stage3` come layer; si disegna con `map(..., "stage2")`	pochi stage corti, tutti nell'editor
Una mappa grande, regioni	stage affiancati in una mappa larga; `map()` e la camera guardano una regione	stage di dimensioni diverse, nessun cambio di dati
Costruito dal codice	il codice scrive le celle con `mset` da una tabella o da una stringa	stage casuali o generati

Da sapere

- Tutti i layer hanno la stessa dimensione. `msize(w, h)` imposta la dimensione di tutti insieme (fino a 4194304 celle).
- Una mappa ha al massimo 8 layer. Anche lo scenario che sta davanti al personaggio (`front`) ne occupa uno.
- Le collisioni seguono il layer che dai a `lib.tiles` : `lib.tiles({ layer = "stage2" })` .
- I flag sono quelli delle tile e sono gli stessi per ogni stage.
- Le celle con numeri da 1024 in su sono segnaposto: il codice li legge (partenza dell'eroe, nemici, l'uscita) e non li disegna.

Crea tre stage con i layer

1. In bm SDK, premi **F3** due volte. Con **Shift+L** e **r** crea tre layer: `stage1` , `stage2` , `stage3` .
2. Dipingi ogni stage nel suo layer. Premi **o** per guardare un solo layer.
3. Dipingi una tile a tua scelta (l'uscita) nella stessa cella di ogni stage, e usala nel codice.
4. Premi **Ctrl+S**.
5. Nel codice, disegna e collidi con il layer dello stage corrente (capitolo 9).

Regioni in una mappa grande

1. Premi **m** per vedere l'intera mappa. Scegli dove inizia ogni stage, in celle.
2. Dipingi lo stage 1 dalla cella (0, 0), lo stage 2 dalla cella (64, 0), e così via.
3. Nel codice, passa la prima cella dello stage a `map(mx, my, ...)` e a `lib.camera` con `bounds` .

I segnaposto nella mappa 2D (un layer `objects` , pennello 16x16) possono segnare la partenza dell'eroe e i nemici.

Attenzione

- La pagina mappa mostra tutti i layer o uno. **[IN SVILUPPO]** Una vista "stage" che mostri insieme i layer di uno stage non esiste. Cosa manca: gruppi di layer nell'editor.

Più stage in un gioco 3D

Cosa fa

`map3d_build` crea un mondo 3D dai layer. Un gioco con più stage 3D crea un mondo nuovo a ogni stage, con i layer di quello stage.

Da sapere

- `map3d_build` prende i nomi dei layer: `{ floor = "floor2", height = "height2", walls = "walls2" }`. Il layer degli oggetti è sempre `objects`.
- La mappa ha al massimo 8 layer. Un secondo stage può avere i propri layer `floor`, `height` e `walls`, ma condivide il layer `objects`.
- Un mondo è un valore del gioco: tieni quello dello stage in cui sei, e costruisci il successivo quando cambi.
- L'editor costruisce il mondo solo con i nomi di default (`floor`, `height`, `walls`).

Le funzioni del gioco

Funzione	Cosa dà
`map3d_build(opts)`	il mondo, dai layer
`map3d_draw(w)`	disegna i blocchi visibili di 16x16 tile
`map3d_height(w, x, z)`	l'altezza del terreno, o `nil` per un buco o fuori
`map3d_blocked(w, x, y, z, r)`	`true` se una sfera tocca un muro o un dirupo
`map3d_objects(w, kind)`	l'elenco degli oggetti, con `kind`, `x`, `y`, `z`
`map3d_size(w)`	la dimensione del mondo

Attenzione

- **[IN SVILUPPO]** Più stage 3D completi in una cartuccia non sono supportati dall'editor. Cosa manca: un layer `objects` per ogni stage (è sempre `objects`) e un'anteprima 3D di un insieme di layer diverso da `floor`, `height` e `walls`. Oggi: scrivi le celle dello stage successivo con `mset` prima di `map3d_build`, oppure distribuisci una cartuccia per stage.
- **[IN SVILUPPO]** Non c'è collisione con i modelli degli oggetti. Cosa manca: i volumi di collisione degli oggetti; usa `map3d_blocked` per la mappa e un tuo controllo per gli oggetti.
- Una mappa con altezze è una griglia: niente pendii e niente archi.

Uno stage 3D nel codice

DA COPIARE

```
local world

function load_stage()
  world = map3d_build({ floor = "floor", height = "height", walls = "walls" })
  for _, o in ipairs(map3d_objects(world, "PLAYER_START")) do
    px, py, pz = o.x, o.y, o.z
  end
end

function _draw()
  zclear()
  camera3d(px, py + 2, pz - 4, 0, -0.3, 60)
  map3d_draw(world)
end
```

NOTA

È l'ordine che usa l'anteprima dell'editor: `zclear`, `camera3d`, poi `map3d_draw`. L'editor imposta anche una luce con `light3d`. Per un gioco 3D completo vedi il volume 3D dell'API.

Passare da un livello all'altro nel gioco

Cosa fa

Un gioco va dal titolo allo stage 1, allo stage 2, e così via, fino a un finale. bmlib ha la piccola macchina che lo fa: `lib.states` .

Da sapere

- Uno stato è una tabella con `enter` , `update` , `draw` e `exit` . `S:go(name, ...)` cambia stato e passa dei valori a `enter` .
- `lib.clear()` ferma ogni tween, timer, script e jingle: chiamala quando inizia un nuovo livello.
- `lib.store(k, v)` tiene un valore nel salvataggio della cartuccia: lo stage migliore, le vite.
- Il gioco può anche tenere uno slot con `save()` e leggerlo con `saved()` .

Un gioco con tre stage

DA COPIARE

```
local lib = require "bmlib"
local STAGES = { "stage1", "stage2", "stage3" }
local n = 1

local S
S = lib.states({
  title = {
    update = function() if btnp("ok") then S:go("play", 1) end end,
    draw = function() cls(0); lib.printc("PREMI A", 160, 0xFFFFFF) end },
  play = {
    enter = function(s, i)
      lib.clear()
      n = i
      lib.tiles({ layer = STAGES[n], edge = true })
      -- metti l'eroe dove c'è la tile di partenza
    end,
    update = function(s)
      lib.step(hero)
      if hero_on_exit() then
        lib.store("stage", n)
        if n < #STAGES then S:go("play", n + 1) else S:go("end") end
      end
    end,
    draw = function(s)
      cls(0)
      map(0, 0, 0, 0, 80, 45, STAGES[n])
      spr(1, hero.x, hero.y)
    end },
  ["end"] = {
    draw = function() cls(0); lib.printc("FINE", 160, 0xFFFFFF) end },
}, "title")

function _update() lib.update(); S:update() end
function _draw() S:draw() end
```

Provalo, passo per passo

1. Dipingi tre layer `stage1`, `stage2`, `stage3` in bm SDK (capitolo 7), ciascuno con del terreno.
2. Incolla il codice qui sopra nella pagina codice (**F2**). Scrivi `hero`, `hero_on_exit` e la partenza dell'eroe per il tuo gioco.
3. Premi **F5**. Al titolo, premi il tasto di conferma: inizia lo stage 1.
4. Raggiungi l'uscita: si carica lo stage 2, con il suo layer, e `lib.clear()` ha fermato i timer del primo.
5. Finisci lo stage 3 per vedere la schermata finale. Esci con Start + Select.

> **ATTENZIONE**
> Se il nome di un layer è sbagliato, la chiamata fallisce con un errore. Controlla il nome con `mlayers()` e tieni nel codice un'unica lista dei nomi.

Attenzione

- **[IN SVILUPPO]** La schermata del titolo, l'elenco degli stage e il finale vanno scritti a mano. Cosa manca: un modello con menu, più stage e un finale. È previsto per la beta (rev 1): il suo test di accettazione chiede un gioco con titolo, 3 stage e finale fatto solo con la suite.
- Per la versione 3D la macchina è la stessa: in `enter`, riempi i layer dello stage con `mset`, chiama `map3d_build` e conserva il mondo.

Consigli e limiti

Limiti

Cosa	Limite
Layer di una mappa	da 1 a 8
Dimensione dello sheet	multipli di 8, fino a 4096
Mappa nuova	256x256 celle, un layer
Mappa impostata con `msize`	fino a 4194304 celle
Flag di una tile	8 (da 0 a 7)
Zone in un file	da 1 a 1024
Box in un file	da 1 a 4096
Lampade in 3D	4 (slot `lamp3d` da 1 a 4)
Larghezza del percorso	da 1 a 5

Consigli

- Disegna le tile a dimensione 8 e tieni insieme sullo sheet le tile dello stesso tipo.
- Dai ai layer nomi che dicano cosa sono, e tieni la lista dei nomi in un solo punto del codice.
- Prova il gioco con **F5** dopo ogni stage. Torni alla stessa pagina.
- In 3D, guarda la stima dei triangoli nella barra di stato mentre aggiungi oggetti.
- Tieni le tile di uscita e di partenza nella stessa cella di ogni layer, per riusare il codice.

Quando qualcosa va storto

- `a map has at most 8 layers` : toglierne uno prima di aggiungerne un altro.
- Un percorso o una fila non è nel layer che ti aspetti: controlla che i layer si chiamino `floor` e `objects` .
- Un pavimento 3D è pieno di buchi: con il pennello 16x16, la tile 0 dà un buco. Scegli una tile diversa da 0.
- Uno stage non collide: controlla il layer che hai passato a `lib.tiles` e che le tile abbiano il flag 0.

Cosa non c'è ancora

- **[IN SVILUPPO]** Un elenco di stage nella cartuccia e nell'editor.
- **[IN SVILUPPO]** `lib.world()` e la griglia di stanze.
- **[IN SVILUPPO]** L'importazione da Tiled.
- **[IN SVILUPPO]** Un editor per i box.
- **[IN SVILUPPO]** Un layer `objects` per ogni stage 3D.
- **[IN SVILUPPO]** Un tasto per `CRATE_BOUNCE` (1034) nell'editor.

Punti aperti

Nessun punto aperto.